

BUỔI 3. DOCKER CƠ BẢN VÀ CONTAINER HÓA ỨNG DỤNG MERN

Thời lượng: 4 giờ

Môi trường: Máy tính cá nhân, Docker Desktop, VS Code

Ứng dụng: Tiếp tục ứng dụng Quản lý sinh viên của Buổi 2

Database: MongoDB Atlas

Công nghệ: Docker, Dockerfile, Docker Compose, Git/GitHub

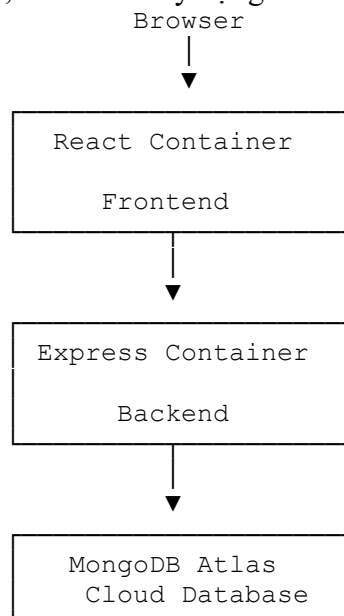
Mục tiêu

Sau khi hoàn thành buổi thực hành, sinh viên có khả năng:

1. Clone Repository của Buổi 2 từ GitHub về máy tính cá nhân và tiếp tục phát triển ứng dụng.
2. Kiểm tra và sử dụng Docker trên môi trường máy local.
3. Thực hiện các thao tác cơ bản với Docker Image và Docker Container.
4. Hiểu cấu trúc và cách hoạt động của Dockerfile.
5. Xây dựng Docker Image cho ứng dụng Node.js/Express.
6. Chạy Backend Express trong Docker Container.
7. Xây dựng Docker Image cho React Frontend.
8. Chạy React Frontend trong Docker Container.
9. Kết nối Frontend Container với Backend Container.
10. Sử dụng Docker Network để giao tiếp giữa các Container.
11. Sử dụng Docker Compose để quản lý ứng dụng MERN gồm Frontend và Backend.
12. Kết nối Backend Container với MongoDB Atlas.
13. Sử dụng Git/GitHub để quản lý và cập nhật source code sau khi container hóa ứng dụng.

Kết quả cần đạt

Cuối buổi, sinh viên xây dựng và chạy được ứng dụng MERN theo mô hình:



Sinh viên có thể giải thích được quy trình:

GitHub Repository → Local → Docker Image → Docker Container → MERN

Application → MongoDB Atlas

và thực hiện các chức năng CRUD sinh viên trên ứng dụng MERN chạy bằng Docker.

PHẦN 1. CLONE REPOSITORY BUỔI 2 VỀ MÁY LOCAL

Câu 1. Mở GitHub và truy cập Repository `cloud-lab` đã hoàn thành ở Buổi 2. (Gợi ý: Mở Repository trên GitHub.)

Câu 2. Sao chép địa chỉ HTTPS của Repository. (Gợi ý: Chọn **Code** → **HTTPS** → **Copy URL**.)

Câu 3. Mở Git Bash hoặc Terminal trên máy tính cá nhân. (Gợi ý: Có thể sử dụng Git Bash hoặc Terminal trong VS Code.)

Câu 4. Kiểm tra phiên bản Git đã được cài đặt. (Gợi ý lệnh: `git --version`)

Câu 5. Di chuyển đến thư mục dùng để lưu các bài thực hành. (Gợi ý lệnh: `cd <đường_dẫn>`)

Câu 6. Clone Repository `cloud-lab` từ GitHub về máy local. (Gợi ý lệnh: `git clone <repository-url>`)

Câu 7. Di chuyển vào thư mục Repository vừa clone. (Gợi ý lệnh: `cd cloud-lab`)

Câu 8. Kiểm tra Remote Repository đã được cấu hình. (Gợi ý lệnh: `git remote -v`)

Câu 9. Kiểm tra Branch hiện tại. (Gợi ý lệnh: `git branch --show-current`)

Câu 10. Kiểm tra trạng thái source code sau khi clone. (Gợi ý lệnh: `git status`)

Câu 11. Liệt kê các file và thư mục trong Repository. (Gợi ý lệnh: `ls -la` hoặc `dir`.)

Câu 12. Mở Repository bằng Visual Studio Code. (Gợi ý lệnh: `code .`)

Câu 13. Kiểm tra các thư mục `client` và `server` của ứng dụng MERN. (Gợi ý: Quan sát cây thư mục trong VS Code.)

Mục tiêu: Sinh viên đưa source code ứng dụng MERN đã hoàn thành ở Buổi 2 từ GitHub về máy local để tiếp tục container hóa bằng Docker.

PHẦN 2. DOCKER CƠ BẢN

Câu 14. Kiểm tra phiên bản Docker đã được cài đặt. (Gợi ý lệnh: `docker --version`)

Câu 15. Kiểm tra thông tin Docker Engine. (Gợi ý lệnh: `docker info`)

Câu 16. Tải Docker Image `hello-world` từ Docker Hub. (Gợi ý lệnh: `docker pull hello-world`)

Câu 17. Liệt kê các Docker Image đang có trên máy. (Gợi ý lệnh: `docker images`)

Câu 18. Tạo và chạy một Docker Container từ Image `hello-world`. (Gợi ý lệnh: `docker run hello-world`)

Câu 19. Liệt kê tất cả các Container đã được tạo. (Gợi ý lệnh: `docker ps -a`)

Câu 20. Tải Docker Image `nginx` từ Docker Hub. (Gợi ý lệnh: `docker pull nginx`)

Câu 21. Tạo và chạy Container từ Image `nginx`. (Gợi ý lệnh: `docker run -d --name nginx-demo nginx`)

Câu 22. Liệt kê các Container đang chạy. (Gợi ý lệnh: `docker ps`)

Câu 23. Xem thông tin Port của Container `nginx-demo`. (Gợi ý lệnh: `docker port nginx-demo`)

Câu 24. Xem log của Container `nginx-demo`. (Gợi ý lệnh: `docker logs nginx-demo`)

Câu 25. Dừng Container `nginx-demo`. (Gợi ý lệnh: `docker stop nginx-demo`)

Câu 26. Khởi động lại Container `nginx-demo`. (Gợi ý lệnh: `docker start nginx-demo`)

Câu 27. Xóa Container `nginx-demo`. (Gợi ý lệnh: `docker rm -f nginx-demo`)

Mục tiêu: Sinh viên hiểu được Docker Image, Docker Container và thực hiện được các thao tác cơ bản với Container.

PHẦN 3. XÂY DỰNG DOCKERFILE CHO BACKEND

Câu 28. Di chuyển vào thư mục `server` của ứng dụng MERN. (Gợi ý lệnh: `cd server`)

Câu 29. Kiểm tra các file trong thư mục Backend. (Gợi ý lệnh: `ls -la`)

Câu 30. Kiểm tra lệnh khởi động Backend trong `package.json`. (Gợi ý: Kiểm tra thuộc tính `scripts`.)

Câu 31. Tạo file Dockerfile trong thư mục Backend. (Gợi ý lệnh: `touch Dockerfile`)

Câu 32. Khai báo Node.js Image làm Base Image cho Backend. (Gợi ý: Sử dụng chỉ thị `FROM`.)

Câu 33. Thiết lập thư mục làm việc bên trong Backend Container. (Gợi ý: Sử dụng chỉ thị `WORKDIR`.)

Câu 34. Sao chép `package.json` và `package-lock.json` vào Container. (Gợi ý: Sử dụng chỉ thị `COPY`.)

Câu 35. Cài đặt các package của Backend trong quá trình Build Docker Image. (Gợi ý: Sử dụng `RUN npm install`.)

Câu 36. Sao chép source code Backend vào Container. (Gợi ý: Sử dụng `COPY . .`.)

Câu 37. Khai báo port 5000 mà Backend sử dụng. (Gợi ý: Sử dụng `EXPOSE 5000`.)

Câu 38. Thiết lập lệnh khởi động Backend khi Container chạy. (Gợi ý: Sử dụng `CMD` với lệnh `npm start`.)

Câu 39. Quay lại thư mục gốc của Repository. (Gợi ý lệnh: `cd ..`)

Câu 40. Xây dựng Docker Image cho Backend. (Gợi ý lệnh: `docker build -t mern-backend ./server`)

Câu 41. Kiểm tra Docker Image Backend vừa được tạo. (Gợi ý lệnh: `docker images`)

Câu 42. Chạy Backend bằng Docker Container và ánh xạ port 5000. (Gợi ý lệnh: `docker run -d --name mern-backend -p 5000:5000 ...`)

Câu 43. Kiểm tra Backend Container đang chạy. (Gợi ý lệnh: `docker ps`)

Câu 44. Xem log của Backend Container. (Gợi ý lệnh: `docker logs mern-backend`)

Câu 45. Kiểm tra API `/api/hello` của Backend đang chạy trong Container. (Gợi ý lệnh: `curl http://localhost:5000/api/hello`)

Câu 46. Kiểm tra API `/api/students` và xác nhận Backend Container có thể kết nối với MongoDB Atlas. (Gợi ý lệnh: `curl http://localhost:5000/api/students`)

Mục tiêu: Sinh viên chuyển Backend từ chạy trực tiếp bằng Node.js sang chạy trong Docker Container.

PHẦN 4. CONTAINER HÓA REACT FRONTEND

Câu 47. Di chuyển vào thư mục `client` của ứng dụng MERN. (Gợi ý lệnh: `cd client`)

Câu 48. Kiểm tra các file của React Application. (Gợi ý lệnh: `ls -la`)

Câu 49. Kiểm tra các lệnh được khai báo trong `package.json`. (Gợi ý: Kiểm tra thuộc tính `scripts`.)

Câu 50. Tạo file Dockerfile trong thư mục `client`. (Gợi ý lệnh: `touch Dockerfile`)

Câu 51. Khai báo Node.js Image làm Base Image cho quá trình Build React Application. (Gợi ý: Sử dụng chỉ thị `FROM`.)

Câu 52. Thiết lập thư mục làm việc bên trong Frontend Container. (Gợi ý: Sử dụng chỉ thị `WORKDIR`.)

Câu 53. Sao chép `package.json` và `package-lock.json` vào Container. (Gợi ý: Sử dụng chỉ thị `COPY`.)

- Câu 54.** Cài đặt các package của React Application. (Gợi ý: Sử dụng `RUN npm install`.)
- Câu 55.** Sao chép source code React vào Container. (Gợi ý: Sử dụng `COPY . .`.)
- Câu 56.** Build React Application trong quá trình xây dựng Docker Image. (Gợi ý: Sử dụng `RUN npm run build`.)
- Câu 57.** Kiểm tra thư mục chứa kết quả Build của React Application. (Gợi ý: Kiểm tra thư mục `dist`.)
- Câu 58.** Xây dựng Docker Image cho Frontend. (Gợi ý lệnh: `docker build -t mern-frontend .`.)
- Câu 59.** Kiểm tra Docker Image Frontend vừa được tạo. (Gợi ý lệnh: `docker images`.)
- Câu 60.** Chạy React Frontend bằng Docker Container. (Gợi ý: Sử dụng `docker run` và ánh xạ port của Container ra port trên máy local.)
- Câu 61.** Kiểm tra Frontend Container đang chạy. (Gợi ý lệnh: `docker ps`.)
- Câu 62.** Truy cập React Frontend bằng trình duyệt trên máy local. (Gợi ý: Truy cập `http://localhost:<port>`.)
- Câu 63.** Kiểm tra giao diện React và xác nhận Frontend đang chạy từ Docker Container. (Gợi ý: Quan sát ứng dụng trên trình duyệt.)
- Mục tiêu:** Sinh viên xây dựng được Docker Image và chạy React Frontend trong Docker Container.
-

PHẦN 5. KẾT NỐI MERN CONTAINER

- Câu 64.** Kiểm tra Backend Container và Frontend Container đang chạy. (Gợi ý lệnh: `docker ps`.)
- Câu 65.** Kiểm tra Port của Backend và Frontend Container. (Gợi ý lệnh: `docker ps`.)
- Câu 66.** Kiểm tra Endpoint Backend đang được sử dụng trong React Frontend. (Gợi ý: Tìm URL API trong source code React.)
- Câu 67.** Tạo Docker Network cho ứng dụng MERN. (Gợi ý lệnh: `docker network create mern-network`.)
- Câu 68.** Kiểm tra Docker Network vừa tạo. (Gợi ý lệnh: `docker network ls`.)
- Câu 69.** Kết nối Backend Container vào Docker Network. (Gợi ý lệnh: `docker network connect mern-network mern-backend`.)
- Câu 70.** Kết nối Frontend Container vào Docker Network. (Gợi ý lệnh: `docker network connect mern-network <frontend-container>`.)
- Câu 71.** Kiểm tra các Container đang thuộc Docker Network `mern-network`. (Gợi ý lệnh: `docker network inspect mern-network`.)
- Câu 72.** Kiểm tra Backend API từ môi trường Docker. (Gợi ý: Sử dụng `curl` để gọi API Backend.)
- Câu 73.** Kiểm tra Backend Container có thể kết nối đến MongoDB Atlas. (Gợi ý: Kiểm tra log bằng `docker logs mern-backend`.)
- Câu 74.** Truy cập Frontend bằng trình duyệt và hiển thị danh sách sinh viên. (Gợi ý: Kiểm tra React gọi API Backend thành công.)
- Câu 75.** Thêm một sinh viên từ giao diện React. (Gợi ý: Sử dụng Form nhập MSSV, Họ tên và Email.)
- Câu 76.** Kiểm tra sinh viên vừa thêm trên MongoDB Atlas. (Gợi ý: Mở Collection `students`.)
- Câu 77.** Thực hiện cập nhật một sinh viên từ giao diện React. (Gợi ý: Sử dụng API `PUT /api/students/:id`.)
- Câu 78.** Thực hiện xóa một sinh viên từ giao diện React. (Gợi ý: Sử dụng API `DELETE /api/students/:id`.)
-

PHẦN 6. DOCKER COMPOSE CHO ỨNG DỤNG MERN

Câu 79. Dừng các Container Backend và Frontend đang chạy. (Gợi ý lệnh: `docker stop ...`)

Câu 80. Tạo file `docker-compose.yml` tại thư mục gốc của Repository. (Gợi ý lệnh: `touch docker-compose.yml`)

Câu 81. Khai báo Service backend trong Docker Compose. (Gợi ý: Sử dụng thuộc tính `services` và `build`.)

Câu 82. Khai báo Service frontend trong Docker Compose. (Gợi ý: Sử dụng thuộc tính `services` và `build`.)

Câu 83. Cấu hình Port cho Service backend. (Gợi ý: Ánh xạ port 5000 của Container ra port trên máy local.)

Câu 84. Cấu hình Port cho Service frontend. (Gợi ý: Ánh xạ port của Frontend Container ra port trên máy local.)

Câu 85. Cấu hình biến môi trường `MONGODB_URI` cho Backend trong Docker Compose. (Gợi ý: Sử dụng thuộc tính `environment`.)

Câu 86. Khởi động toàn bộ ứng dụng MERN bằng Docker Compose. (Gợi ý lệnh: `docker compose up -d --build`)

Câu 87. Kiểm tra trạng thái các Service đang chạy. (Gợi ý lệnh: `docker compose ps`)

Câu 88. Xem log của Backend Service. (Gợi ý lệnh: `docker compose logs backend`)

Câu 89. Truy cập React Frontend bằng trình duyệt. (Gợi ý: Truy cập `http://localhost:<port>`.)

Câu 90. Kiểm tra chức năng lấy danh sách sinh viên từ MongoDB Atlas. (Gợi ý: Kiểm tra luồng `React → Backend → MongoDB Atlas`.)

Câu 91. Thêm một sinh viên và kiểm tra dữ liệu trên MongoDB Atlas. (Gợi ý: Sử dụng Form trên React.)

Câu 92. Dừng toàn bộ hệ thống MERN bằng Docker Compose. (Gợi ý lệnh: `docker compose down`)

Câu 93. Khởi động lại toàn bộ hệ thống bằng Docker Compose. (Gợi ý lệnh: `docker compose up -d`)

Câu 94. Kiểm tra lại ứng dụng sau khi khởi động lại các Container. (Gợi ý: Sử dụng `docker compose ps` và truy cập Frontend bằng trình duyệt.)

Mục tiêu: Sinh viên sử dụng Docker Compose để xây dựng và quản lý đồng thời Frontend Container và Backend Container.

PHẦN 7. GIT VÀ HOÀN THÀNH BÀI THỰC HÀNH

Câu 95. Kiểm tra các file đã thay đổi sau khi hoàn thành Buổi 3. (Gợi ý lệnh: `git status`)

Câu 96. Kiểm tra `.gitignore` và đảm bảo file `.env` không được đưa vào Git. (Gợi ý lệnh: `git status`)

Câu 97. Kiểm tra các file Docker trước khi commit. (Gợi ý: Kiểm tra `client/Dockerfile`, `server/Dockerfile` và `docker-compose.yml`.)

Câu 98. Đưa source code và cấu hình Docker vào Staging Area. (Gợi ý lệnh: `git add .`)

Câu 99. Tạo commit cho kết quả thực hành Buổi 3. (Gợi ý lệnh: `git commit -m "Complete Lab 03 Docker MERN"`)

Câu 100. Push source code lên GitHub. (Gợi ý lệnh: `git push`)

PHẦN 8. BÀI TẬP TỔNG HỢP

Câu 101. Dừng toàn bộ các Container của ứng dụng MERN. (Gợi ý lệnh: `docker compose down`)

Câu 102. Khởi động lại toàn bộ ứng dụng MERN bằng Docker Compose. (Gợi ý lệnh: `docker compose up -d --build`)

Câu 103. Kiểm tra trạng thái Backend Container và Frontend Container. (Gợi ý lệnh: `docker compose ps`)

Câu 104. Truy cập Frontend bằng trình duyệt và hiển thị danh sách sinh viên. (Gợi ý: Truy cập ứng dụng thông qua `localhost`.)

Câu 105. Thêm ít nhất 3 sinh viên thông qua giao diện React. (Gợi ý: Sử dụng Form nhập dữ liệu.)

Câu 106. Thực hiện cập nhật một sinh viên thông qua ứng dụng. (Gợi ý: Sử dụng API `PUT /api/students/:id`.)

Câu 107. Thực hiện xóa một sinh viên thông qua ứng dụng. (Gợi ý: Sử dụng API `DELETE /api/students/:id`.)

Câu 108. Kiểm tra log Backend Container và xác nhận các Request từ Frontend đã được xử lý. (Gợi ý lệnh: `docker compose logs backend`)

Câu 109. Kiểm tra Repository trên GitHub và xác nhận source code Docker đã được Push thành công. (Gợi ý: Kiểm tra `client/Dockerfile`, `server/Dockerfile` và `docker-compose.yml`.)